

OF-3 Report :

Simulating Supersonic Flow over a step

Terry Murray (tnm3843), Nicole Olvera (no4342), Andres Suniaga (aas5778)

04/10/2025

Contents

1	Introduction	2
2	Governing Equations	3
2.1	Two-dimensional Euler equations	3
2.2	Local Sound Speed and Mach Number	4
2.3	Normal Shock and Isentropic Flow Relations	4
2.4	Step Example	5
3	First Solution	6
4	Converging the Solution	8
4.1	Mesh hierarchy and time step	8
4.2	Grid-convergence of key variables	8
4.3	Pressure distribution along surfaces	10
4.4	Discussion of convergence	11
5	Parallel Solver Performance	12
A	Code	

1 Introduction

This report entails simulations of Mach 3 supersonic flow over a step as shown in Figure 1. We first take a walk through the equations that govern this flow such as a simplified *Navier-Stokes* equations which make up Euler's equations and Normal Shock and Isentropic Flow theory.

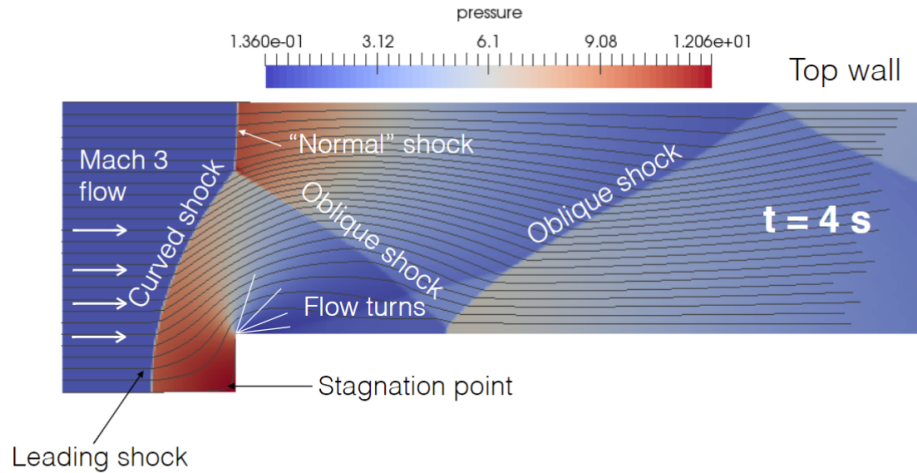


Figure 1: Details of the flow studied

We then simulate this flow using *OpenFoam's rhoCentralFoam* solver for various meshes from a coarse solution to more refined grids. *Paraview* is used to visualize the flow features such as the Mach number, velocity, temperature, pressure, and density. The stagnation point on the step is a point of interest as we compare the result of theory and CFD for the total pressure at this location. Lastly, we evaluate the performance of the parallel solver and observe how the number of processors affects the efficiency of the solver.

2 Governing Equations

2.1 Two-dimensional Euler equations

We consider the Euler equations in $\mathbb{R}^2$ complemented by the equation of state of an ideal gas (Eq. 6) with the specific heat ratio $\gamma = 1.4$. The two-dimensional Euler equations read:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} + \frac{\partial \mathbf{G}}{\partial y} = 0 \quad (1)$$

Where $\mathbf{U}$ is the state vector, $\mathbf{F}$ is the x-flux vector, and $\mathbf{G}$ is the y-flux vector.

From the assumptions of Euler equations, e.g., no body forces and inviscid and isothermal flow, the simplified form *Navier-Stokes Equations* read:

- Continuity :

$$\frac{\partial \rho}{\partial t} + \rho \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right) = 0 \quad (2)$$

- X-Momentum :

$$\frac{\partial \rho u}{\partial t} + \rho \left(\frac{\partial u^2}{\partial x} + \frac{\partial uv}{\partial y} \right) = -\frac{\partial p}{\partial x} \quad (3)$$

- Y-Momentum :

$$\frac{\partial \rho v}{\partial t} + \rho \left(\frac{\partial uv}{\partial x} + \frac{\partial v^2}{\partial y} \right) = -\frac{\partial p}{\partial y} \quad (4)$$

- Energy (inviscid simplification):

$$\frac{\partial E}{\partial t} + E \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right) = -p \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right) \quad (5)$$

Where $E = \rho(e + ||\mathbf{u}||^2/2)$. e is the internal energy of the fluid per unit mass (Joule per kilogram in SI).

Thus the state vector $\mathbf{U} = \begin{bmatrix} \rho \\ \rho u \\ \rho v \\ \rho(e + \frac{1}{2}(u^2 + v^2)) \end{bmatrix} = \begin{bmatrix} \rho \\ \rho u \\ \rho v \\ E \end{bmatrix}.$

The x-flux vector $\mathbf{F} = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ (\rho(e + \frac{1}{2}(u^2 + v^2)) + p) * u \end{bmatrix} = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ (E + p)(u) \end{bmatrix}$

The y-flux vector $\mathbf{G} = \begin{bmatrix} \rho v \\ \rho vu \\ \rho v^2 + p \\ (\rho(e + \frac{1}{2}(u^2 + v^2)) + p)(v) \end{bmatrix} = \begin{bmatrix} \rho v \\ \rho vu \\ \rho v^2 + p \\ (E + p)(v) \end{bmatrix}$

The *Ideal Gas Law* is our equation of state that relates the variables, ρ which is our fluids density, p the local pressure, R the molecular gas constant, and T the local temperature.

$$p = \rho RT \quad (6)$$

Using the first law of Thermodynamics, p can be related to the specific internal energy e and the specific heat ratio γ by:

$$e = \frac{p}{\rho(\gamma - 1)} \quad (7)$$

These can be combined for a new expression that expresses p entirely in terms of the state vector $\mathbf{U}$, allowing the flux vectors $\mathbf{F}(\mathbf{U})$ and $\mathbf{G}(\mathbf{U})$ to be evaluated directly from conserved quantities.

$$p = (\gamma - 1) \left(E - \frac{1}{2} \rho (u^2 + v^2) \right) \quad (8)$$

2.2 Local Sound Speed and Mach Number

To analyze compressible flow behavior, we often need the local sound speed and Mach number.

Local Sound Speed:

The local speed of sound a in a compressible fluid is given by the equation:

$$a = \sqrt{\gamma RT} = \sqrt{\gamma \frac{p}{\rho}} \quad (9)$$

Local Mach Number:

The local Mach number M is defined as the ratio of the flow speed to the local speed of sound:

$$M = \frac{\sqrt{u^2 + v^2}}{a} = \frac{|\mathbf{U}|}{a} \quad (10)$$

where u and v are the flow velocity components in the x and y directions, respectively.

2.3 Normal Shock and Isentropic Flow Relations

Here we look at the equations for normal shocks and isentropic flows, i.e., adiabatic and reversible.

For a normal shock with upstream (region 1) Mach number M_1 and downstream (region 2) Mach number M_2 , the following standard jump conditions apply (for a perfect gas with ratio of specific heats γ):

- **Static Pressure Ratio:**

$$\frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma + 1} (M_1^2 - 1). \quad (11)$$

where:

- p_1 : upstream (pre-shock) static pressure,

– p_2 : downstream (post-shock) static pressure.

• **Downstream Mach Number:**

$$M_2^2 = \frac{1 + \frac{\gamma-1}{2} M_1^2}{\gamma M_1^2 - \frac{\gamma-1}{2}}. \quad (12)$$

Once the flow is on either side of a shock (and assuming no further shocks/viscous effects in that region), it can be treated as isentropic. In an isentropic flow of a perfect gas:

• **Isentropic Pressure Relation:**

$$p_0 = p \cdot \left(1 + \frac{(\gamma-1)}{2} M_1^2\right)^{\frac{\gamma}{\gamma-1}} \quad (13)$$

where:

- p_0 : total (stagnation) pressure,
- p : static pressure at a given point in the flow,
- M : local Mach number at that point.

2.4 Step Example

Considering a flow over our step at $M = 3$ and $p = 1$ Pa, what is the pressure at the stagnation point ($x=0.6\text{m}$, $y=0\text{m}$ in Figure 2) on the surface of the step? We will employ the equations laid out above.

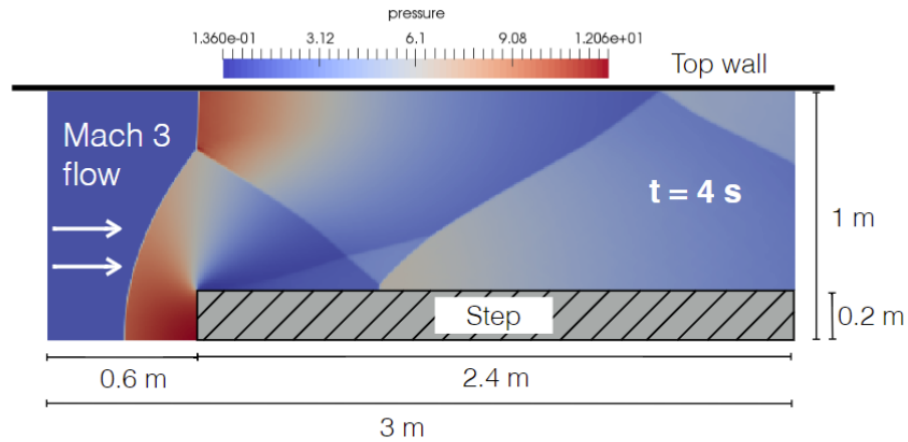
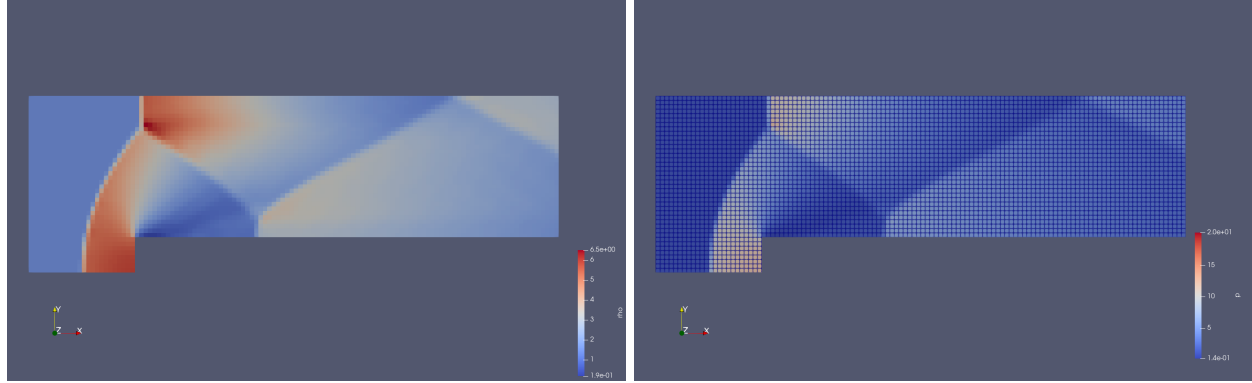


Figure 2: Overview of the Flow

Using Equation 13 we can obtain the total pressure in the first region $p_{0_1} = 36.73$ Pa. We can also obtain the local Mach past the shock using Equation 12, $M_2 = 0.4752$. We can obtain our static pressure past the shock as $p_2 = p_1 * \text{Eq. 11} = 10.33$ Pa. Thus using Equation 13 again, we get a stagnation pressure $p_{0_2} = 12.06$ Pa.

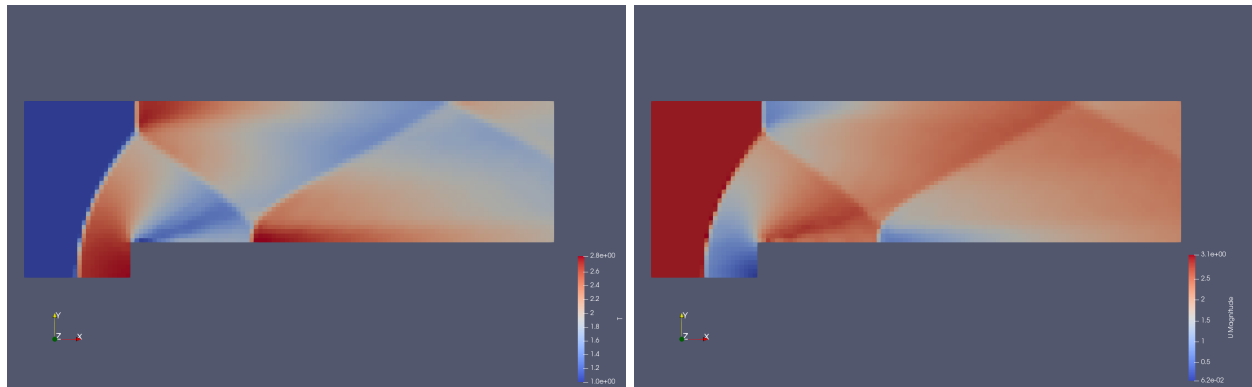
3 First Solution

In this section we visualize the properties pressure p , density ρ , velocity $\mathbf{U}$, and temperature T after simulating Mach 3 flow over the step for $T = 4$ s.



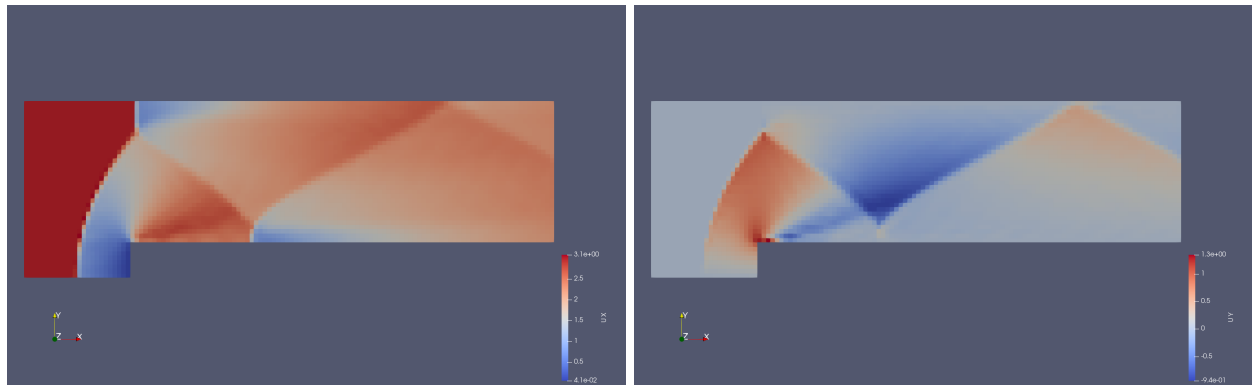
(a) Density ρ

(b) Pressure p



(c) Temperature T

(d) Magnitude of Velocity $|\mathbf{U}|$



(e) x-velocity u

(f) y-velocity v

Figure 3: Coarse mesh solution, gridpoints can be seen in the pressure plot

Using the built-in calculator in *Paraview*, we are able to employ Equations 9 and 10 to visualize the Mach and Speed of Sound in our supersonic simulation.

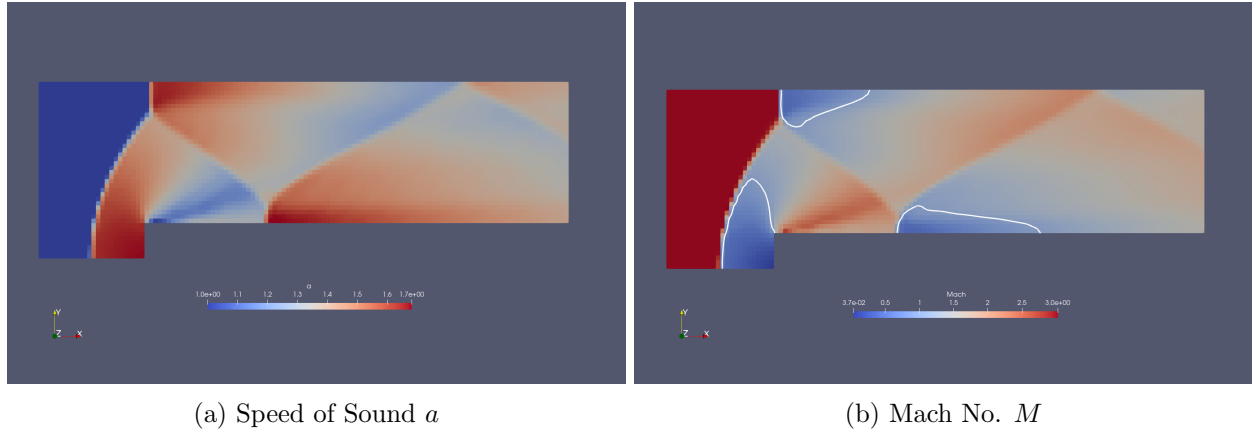


Figure 4: White countour lines representing sonic flow, i.e., where $M = 1$

The pressure at the stagnation grid point in this coarse mesh is $p = 11.9647$ Pa. This is close but slightly less than what was computed using our normal shock theory.

4 Converging the Solution

4.1 Mesh hierarchy and time step

MeshID	Cells	Δt [s]
Nx1	4,032	0.006
Nx2	16,128	0.019
Nx4	64,512	0.076
Nx8	258048	0.009

Table 1: Mesh and solver parameters used in the convergence study.

NOTE: Decrease in delta T for Nx8 mesh since that mesh was ran in parallel on 56 processors.

4.2 Grid-convergence of key variables

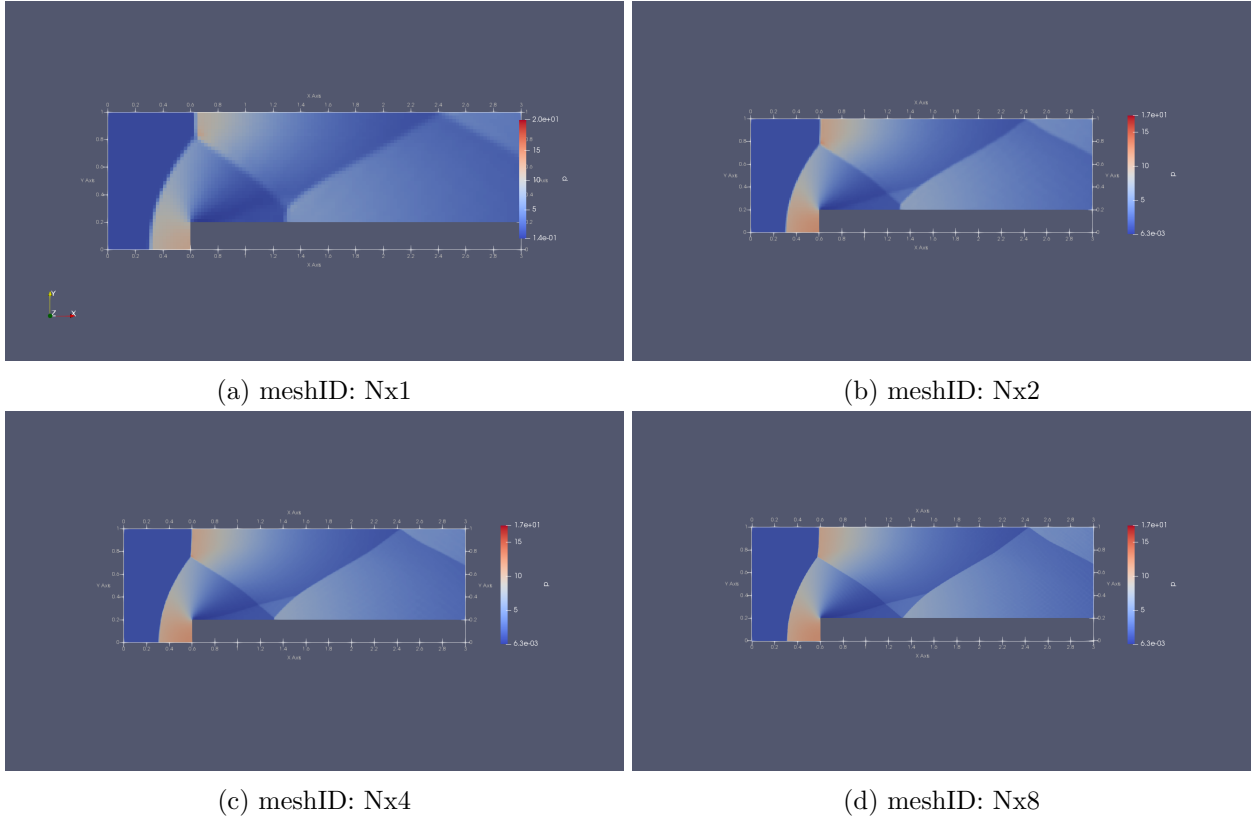
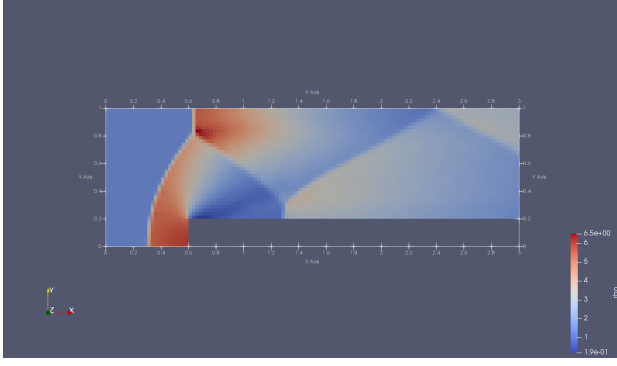
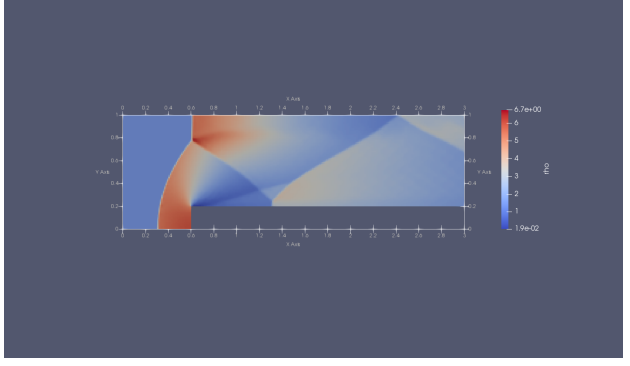


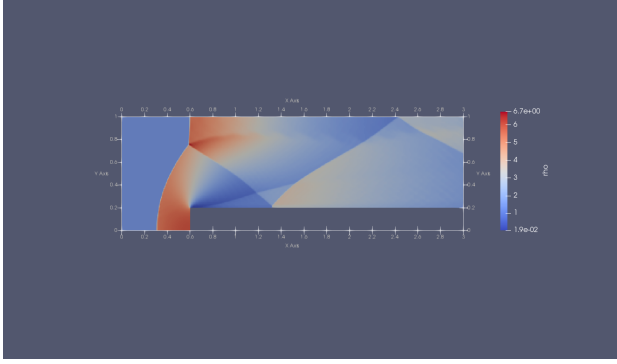
Figure 5: Static pressure field in [Pa] at $t = 4$ s for meshes Nx1, Nx2, Nx4, Nx8. All panels share identical contour limits (0.1 Pa - 17 Pa) to highlight sharpening of the shock with refinement.



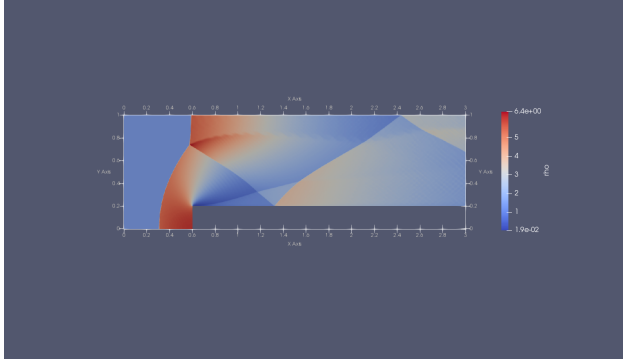
(a) meshID: Nx1



(b) meshID: Nx2



(c) meshID: Nx4

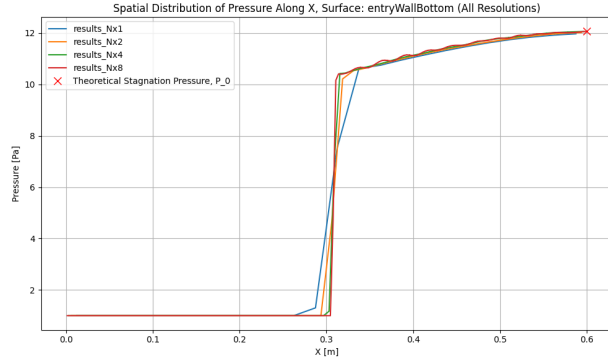


(d) meshID: Nx8

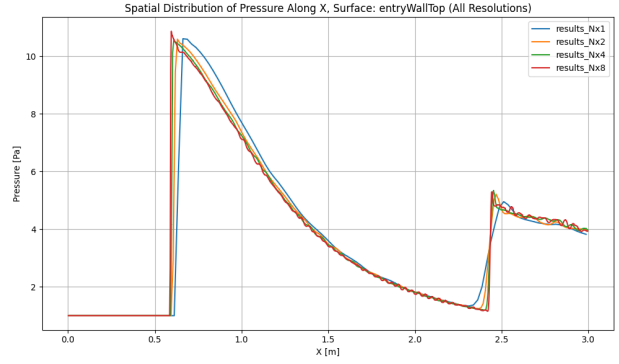
Figure 6: Static density field in $[\text{kg m}^{-3}]$ at $t = 4$ s for meshes Nx1, Nx2, Nx4, Nx8. All panels use identical contour limits to emphasise convergence of density gradients across the normal shock and expansion fan.

In the following sections the pressure is recorded along a line very close to the surfaces (offset $1e-5$ meters) across the span of every surface.

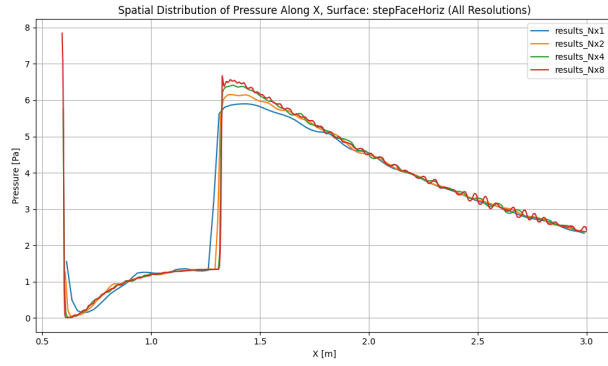
4.3 Pressure distribution along surfaces



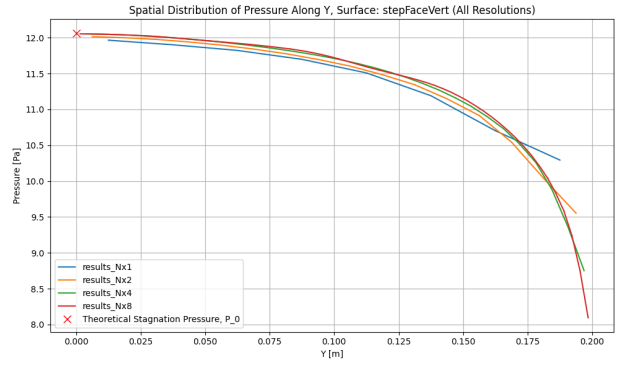
(a) Bottom wall (inlet floor)



(b) Top wall (tunnel ceiling)

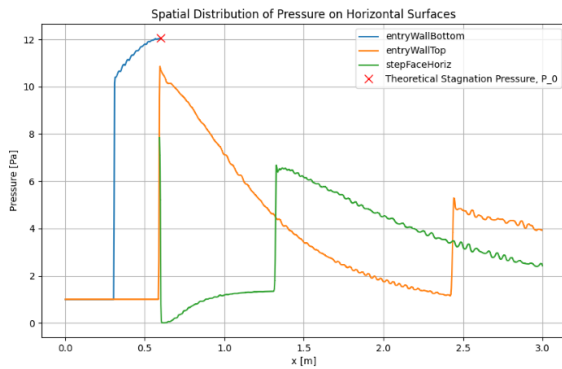


(c) Step face — horizontal

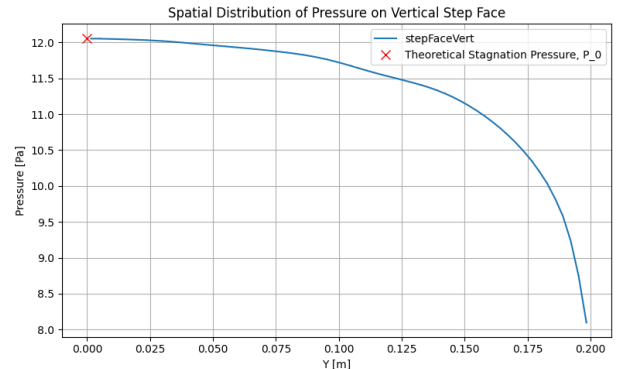


(d) Step face — vertical

Figure 7: Wall-pressure profiles at $t = 4$ s for meshes Nx1 (blue), Nx2 (orange), Nx4 (green), Nx8 (red). The red “x” denotes the theoretical stagnation pressure $p_{0,2} = 12.06$ Pa.



(a) Horizontal walls



(b) Vertical walls

Figure 8: Converged (Nx8) pressure distribution on horizontal (left) and vertical (right) walls at $t = 4$ s.

4.4 Discussion of convergence

The leading-edge stagnation pressure on the bottom wall (Fig. 7, upper-left) climbs from 11.3 Pa on the coarsest grid to 11.96 Pa on Nx8, only $\left| \frac{11.96 - 12.06}{12.06} \right| \approx 0.8\%$ below the analytical value.

Away from the step, the pressure and density fields exhibit convergence: peak-to-peak differences becoming smaller and smaller (compare Figs. 5 and 6).

Overall, the Nx4 mesh provides very high accuracy, while Nx8 serves as a reference solution for quantitative validation. All subsequent comparisons in this report therefore use the Nx8 data set.

5 Parallel Solver Performance

In this section, we analyze the performance characteristics of the parallel version of the `rhoCentralFoam` solver. The primary objective is to evaluate how effectively the solver scales when computing one simulation time step on different numbers of processors. Ideally, as we increase the number of processors, $T(P)$, the time taken by the solver to advance one time step should decrease proportionally. We employ two critical performance metrics: parallel speedup and parallel efficiency.

Parallel speedup is defined as:

$$\text{Speedup} = \frac{T(1)}{T(P)} \quad (14)$$

where $T(P)$ represents the computational time required by the solver to advance the solution over one time step with P processors, and $T(1)$ is the baseline computational time.

Parallel efficiency, η , is defined as:

$$\eta = P^{-1} \frac{T(1)}{T(P)} \quad (15)$$

indicating how effectively processor resources are being utilized.

We conducted scaling tests on a mesh of approximately one million cells, varying the number of processors from 1 up to 56 processors. The results of these scaling tests are summarized in Table 2 below, which lists the measured time step $T(P)$, calculated parallel speedup, and parallel efficiency for each tested processor P . Additionally, Figure 9 illustrates the observed speedup alongside ideal (linear) scaling and theoretical lines representing 80% and 60% parallel efficiency.

Processors (P)	Time $T(P)$ [s]	Speedup	Efficiency (η) [%]
1	1.4183	1.0000	100.00
2	0.6595	2.1505	107.53
4	0.3357	4.2239	105.60
8	0.1764	8.0392	100.49
16	0.1061	13.3668	85.54
32	0.0663	21.3712	66.79
56	0.0529	26.8098	47.87

Table 2: Parallel Performance Metrics Using Different # of Processors

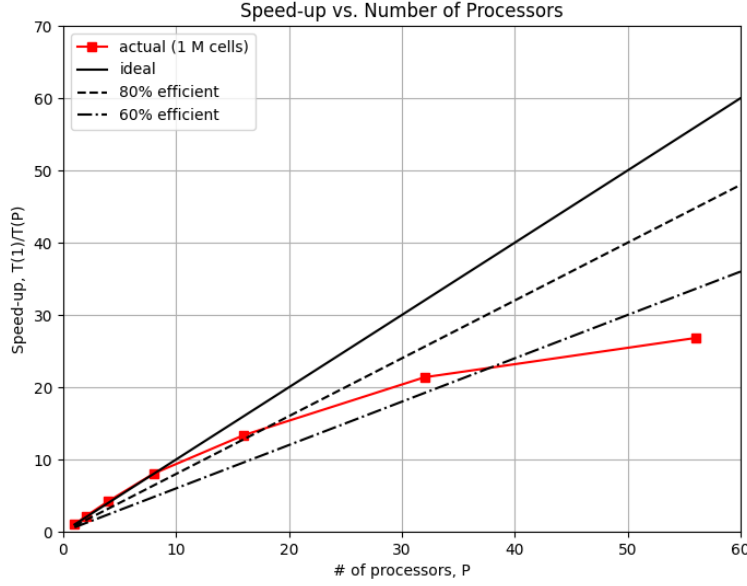


Figure 9: Speedup vs. Number of Processors P .

The collected data and graph clearly demonstrate that initially, as the number of processors increases, the speedup closely follows the ideal linear scaling, indicating efficient parallelization. This nearly linear behavior occurs because the computational workload is sufficiently large to ensure that each processor remains fully utilized, with minimal communication overhead and synchronization requirements. However, as the number of processors continues to increase beyond a certain point (around 16-32 processors in our tests), we observe diminishing returns in the speedup curve. This slowdown in parallel performance may occur due to several factors including: communication overhead, more processors need an increase in communication between them resulting in additional computational overhead, and synchronization delays, as processors counts rise, synchronizing the solver across multiple process introduces small yet accumulating errors.

This analysis highlights the importance of identifying an optimal number of processors to maximize computational efficiency. While adding processors initially leads to significant speed improvements, the overhead and inherent limitations eventually offset these gains. Thus, performance characterization, as demonstrated by our data, plays a crucial role in practical decision-making for deploying parallel computations effectively.

Appendix

A Code

PDF of code starts on next page.

Additionally, team repository: <https://github.com/andressuniaga/CFD-SP25-11>

Theoretical Example

```
import math as m
# ----- #

def normalshockrelations(M,gamma,cp,R):
    '''
    Return : m1,m2,rhoratio,pratio,Tratio,p0ratio
    '''

    m1 = M

    #Mach Relation M2
    m2 = m.sqrt( (1+0.5*(gamma-1)*M**2)/(gamma*M**2 - 0.5*(gamma-1)))

    #Density Relation
    rhoratio = ((gamma+1)*M**2)/(2+(gamma-1)*M**2)

    #Pressure Relation
    pratio = 1 + ((2*gamma)*(gamma+1)**-1)*(M**2 - 1)

    #Temperature Relation
    Tratio = (1-gamma+2*gamma*M**2)*(2+(gamma-1)*M**2)/(
((gamma+1)**2)*M**2 )

    #Entropy Change
    dels = cp*m.log(Tratio) - R*m.log(pratio)

    #stagnation pressure ratio
    p0ratio = m.exp(-(dels)/R)

    # #P01/P1
    # p01p1ratio = (1+ 0.5*(gamma-1)*M**2)**(gamma/gamma-1)

    # #P01/P2
    # p02p2ratio = (1+ 0.5*(gamma-1)*m2**2)**(gamma/gamma-1)

    return m1,m2,rhoratio,pratio,Tratio,p0ratio


def isentropic_pressure(p,M,gamma):
```



```

    return p*(1 + 0.5*(gamma-1)*M**2)**(gamma/(gamma-1))

M1 = 3
p1 = 1 # Pa (N/m^2)
gamma = 1.4
R = 287.05 # J/(kgK)
cp = gamma*R/(gamma-1)

result = normalshockrelations(M1,gamma,cp,R)

p01 = isentropic_pressure(p1,M1,gamma)

print(f"p_01 = {p01} Pa")

M2 = result[1]
p2 = result[3]*p1

# p02 = p01*result[5]

p02 = p2*(1 + 0.5*(gamma-1)*M2**2)**(gamma/(gamma-1))

print(f"M2 = {M2}")

print(f"p2 = {p2} Pa")

print(f"p_02 = {p02} Pa")

```

Converging the Solution

```
import os
import glob
import numpy as np
import matplotlib.pyplot as plt

def plot_coord_vs_pressure_across_res(results_all, selected_prefix,
    coord='x'):
    """
    Plots a chosen coordinate (x or y) vs pressure for a specified
    prefix,
    gathering data from different resolution runs (e.g., Nx1, Nx2, Nx4,
    ...).

    Parameters:
    results_all (dict): Dictionary with keys like "results_Nx1",
    "results_Nx2", etc.
        Each value is a sub-dictionary where keys are
        prefixes and
        values are numpy arrays with columns [x, y, p, T,
        u_x, u_y].
    selected_prefix (str): The prefix key of interest (e.g.,
    "entryWallBottom").
    coord (str): Coordinate to plot against pressure. Should be 'x' or
    'y' (default 'x').
    """
    # Determine which column index to use for the coordinate: 0 for x, 1
    for y.
    coord = coord.lower()
    if coord not in ['x', 'y']:
        raise ValueError("The coordinate must be either 'x' or 'y'")

    coord_index = 0 if coord == 'x' else 1

    plt.figure(figsize=(10, 6))

    # Iterate through each resolution result
    for res_key, res_data in results_all.items():
        if selected_prefix in res_data:
            arr = res_data[selected_prefix]
            # Extract coordinate values and pressure values:
```

```

        coord_vals = arr[:, coord_index]    # x if coord_index=0 else y
        pressure = arr[:, 2]                # pressure is the 3rd column
(index 2)

        # Label based on resolution (e.g., "results_Nx1")
        plt.plot(coord_vals, pressure, marker=',', linestyle='-',
label=res_key)
    else:
        print(f"Prefix '{selected_prefix}' not found in {res_key}")

    plt.xlabel(f"{coord.upper()} [m]")
    plt.ylabel('Pressure [Pa]')
    plt.plot(0.6, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8)    # Example experimental data point
    plt.title(f"Spatial Distribution of Pressure Along {coord.upper()},
Surface: {selected_prefix} (All Resolutions)")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def gather_data(folder_path):
    """
    Reads pairs of files named <prefix>_p_T.xy and <prefix>_U.xy
    from a given folder. Extracts columns:
    x, y, p, T, u_x, u_y
    and returns a dictionary keyed by the prefix, with values
    being a numpy array of shape (N, 6).
    """
    # Find all files that end with '_p_T.xy'
    pT_files = glob.glob(os.path.join(folder_path, '*_p_T.xy'))

    # Dictionary to hold combined data for each prefix
    data_dict = {}

    # Loop over each p_T file
    for pT_file in pT_files:
        # Derive a matching U-file name by replacing '_p_T.xy' with '_U.xy'
        prefix = pT_file.replace('_p_T.xy', '')
        u_file = prefix + '_U.xy'

        # Check if the matching U-file exists
        if os.path.isfile(u_file):

```

```

# Load the p_T file
# Columns: x, y, z, p, T -> indices: 0,1,2,3,4
try:
    pT_data = np.loadtxt(pT_file)
except Exception as err:
    print(f"Error reading {pT_file}: {err}")
    continue

# Load the U file
# Columns: x, y, z, u_x, u_y, u_z -> indices: 0,1,2,3,4,5
try:
    u_data = np.loadtxt(u_file)
except Exception as err:
    print(f"Error reading {u_file}: {err}")
    continue

# Check if both files have same number of rows
if pT_data.shape[0] != u_data.shape[0]:
    print(f"Warning: row mismatch between {pT_file} and
{u_file}")
    # You could decide to skip or handle partial matches
    # differently
    continue

# Extract the columns you care about:
# x = pT_data[:, 0]
# y = pT_data[:, 1]
# p = pT_data[:, 3]
# T = pT_data[:, 4]
# u_x = u_data[:, 3]
# u_y = u_data[:, 4]
# (We are ignoring z and u_z based on your requirement)

combined_data = np.column_stack((
    pT_data[:, 0], # x
    pT_data[:, 1], # y
    pT_data[:, 3], # p
    pT_data[:, 4], # T
    u_data[:, 3],  # u_x
    u_data[:, 4]   # u_y
))

# Store in dictionary keyed by prefix (basename without

```

```

suffixes)
    # For example, if pT_file is 'entryWallBottom_p_T.xy',
    # prefix = 'entryWallBottom'
    # data_dict['entryWallBottom'] = combined_data
    base_name = os.path.basename(prefix)
    data_dict[base_name] = combined_data

else:
    print(f"No matching U file found for {pT_file}")

return data_dict

def plot_pressure_vs_coordinates(data_dict, prefixes=None):
    """
    Plots x vs pressure and y vs pressure for specified prefixes.

    Parameters:
    data_dict (dict): Dictionary with keys as prefix strings and values
    as numpy arrays
                        with columns [x, y, p, T, u_x, u_y].
    prefixes (list, optional): List of prefix keys to plot. If None, all
    keys are plotted.
    """
    if prefixes is None:
        prefixes = list(data_dict.keys())

    # Plot x vs pressure
    plt.figure(figsize=(10, 6))
    for prefix in prefixes:
        data = data_dict[prefix]
        x = data[:, 0]    # x-values
        p = data[:, 2]    # pressure (3rd column)
        plt.plot(x, p, marker=',', linestyle='-', label=prefix)
        plt.xlabel('x [m]')
        plt.ylabel('Pressure [Pa]')
        plt.title('Spatial Distribution of Pressure on Horizontal Surfaces')
        plt.plot(0.6, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8)    # Example experimental data point
        plt.legend()
        plt.grid(True)

    # Plot y vs pressure
    plt.figure(figsize=(10, 6))

```

```

for prefix in prefixes:
    data = data_dict[prefix]
    y = data[:, 1]    # y-values
    p = data[:, 2]    # pressure (3rd column)
    plt.plot(y, p, marker=',', linestyle='-', label=prefix)
    plt.xlabel('y [m]')
    plt.ylabel('Pressure [Pa]')
    plt.title('Spatial Distribution of Pressure on Vertical Surfaces')
    plt.plot(0.6, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8) # Example experimental data point
    plt.legend()
    plt.grid(True)

plt.show()

def truncate_stepFaceVert(results_all, y_threshold=0.2):
    """
    For each resolution (key) in results_all, if the prefix
    "stepFaceVert" exists,
        truncate its data so that only rows with y <= y_threshold are kept.

    Parameters:
        results_all (dict): Dictionary with keys (like "results_Nx1") mapping
to sub-dictionaries
                                where each sub-dictionary has keys for each prefix
and values as numpy arrays
                                with columns [x, y, p, T, u_x, u_y].
        y_threshold (float): The y-value threshold above which the data is
truncated.

                                (Default is 0.2)
    """
    for res_key, data_dict in results_all.items():
        if "stepFaceVert" in data_dict:
            data = data_dict["stepFaceVert"]
            # Option 1: If the data is sorted by y in increasing order:
            idx = np.searchsorted(data[:, 1], y_threshold, side='right')
            truncated_data = data[:, idx, :]

            # Option 2: (Alternate) if you want to filter every row with y
> threshold,
            # uncomment the next two lines and comment the searchsorted
lines above.
            # truncated_data = data[data[:, 1] <= y_threshold, :]

```

```

        # idx = truncated_data.shape[0]

        data_dict["stepFaceVert"] = truncated_data
        print(f"{res_key}: 'stepFaceVert' truncated from
{data.shape[0]} to {truncated_data.shape[0]} rows (y <= {y_threshold}).")
    else:
        print(f"{res_key}: 'stepFaceVert' not found.")

def plot_truncated_stepFaceVert_single_res(results_all, resolution_key,
coord='y'):
    """
    Plots the truncated 'stepFaceVert' data for a given resolution.

    Parameters:
    results_all (dict): Dictionary with keys like "results_Nx1",
"results_Nx2", etc.
    resolution_key (str): The key for the resolution to plot (e.g.,
"results_Nx1").
    coord (str): Which coordinate to plot against Pressure ('x' or 'y').
Default is 'y'.
    """
    if resolution_key not in results_all:
        print(f"Resolution key {resolution_key} not found.")
        return

    data_dict = results_all[resolution_key]
    if "stepFaceVert" not in data_dict:
        print("stepFaceVert data not found for", resolution_key)
        return

    # Get the truncated data
    data = data_dict["stepFaceVert"]
    if data.size == 0:
        print("Truncated data for stepFaceVert is empty for", resolution_key)
        return

    # Determine which column to use: 0 for x, 1 for y.
    coord = coord.lower()
    if coord not in ['x', 'y']:
        raise ValueError("Coordinate must be 'x' or 'y'")

    coord_index = 0 if coord == 'x' else 1

```

```

# Extract coordinate and pressure (pressure is always column 2)
coord_vals = data[:, coord_index]
pressure = data[:, 2]

plt.figure(figsize=(8, 5))
plt.plot(coord_vals, pressure, marker=',', linestyle='-',
label='stepFaceVert')
plt.xlabel(f"{coord.upper()} [m]")
plt.ylabel("Pressure [Pa]")
plt.title('Spatial Distribution of Pressure on Vertical Step Face')
plt.plot(0.0, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8) # Example experimental data point
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

Res = [1, 2, 4, 8]

results_all = {}

for r in range(len(Res)):
    # Example usage:
    folder_path = rf"C:\Users\TerryM\Documents\University of Texas\Intro
to
CFD\OF-3\mach-3-wind-tunnel-with-step-Nx{Res[r]}\postProcessing\singleGraph
\4" # Update to your directory
    results = gather_data(folder_path)
    print("results keys:", list(results.keys()))
    # Save results with key e.g., 'results_Nx1'
    results_all[f"results_Nx{Res[r]}"] = results

# 'results' is a dictionary keyed by prefix, e.g.:
# {
#     "entryWallBottom": array([[x1, y1, p1, T1, u_x1, u_y1],
#                                [x2, y2, p2, T2, u_x2, u_y2],
#                                ... ]),
#     "entryWallTop":      array(...),
#     ...
# }

```



```
# Assuming results_all is your dictionary containing the data for different
resolutions:
```

```
truncate_stepFaceVert(results_all, y_threshold=0.2)
```

```
# Example usage for a specific resolution (say, Nx1):
```

```
plot_truncated_stepFaceVert_single_res(results_all, "results_Nx8",
coord='y')
```

```
results = gather_data(folder_path)
```

```
print("results keys:", list(results_all.keys()))
```

```
# You can now access the data for each prefix:
```

```
for res_key, data_dict in results_all.items():
    print(f"\n{res_key}:")
    for prefix, arr in data_dict.items():
        print(f"Prefix: {prefix}, Data shape: {arr.shape}")
        # arr has columns: [x, y, p, T, u_x, u_y]
        # e.g. show the first row
        print("First row:", arr[0, :])
```

```
# Example usage:
```

```
if __name__ == '__main__':
```

```
    # Assume 'data_dict' holds your assembled data from the previous
    step.
```

```
    # For example, data_dict might be structured as:
```

```
    # {
    #     'entryWallBottom': np.array([...]),
    #     'entryWallTop': np.array([...]),
    #     'stepFaceHoriz': np.array([...]),
    #     ...
    # }
```

```
    # If you want to plot all prefixes, just pass data_dict.
```

```
    # To only plot selected prefixes, update the list e.g.,
```

```
['entryWallBottom', 'entryWallTop']
```

```
    selected_prefixes = ['stepFaceVert']
```

```
    # Call the function with your dictionary and the selected prefixes.
```

```
    plot_pressure_vs_coordinates(results, selected_prefixes)
```

```
# Example usage:
```

```
if __name__ == '__main__':
```

```

    # Assuming you have built a results_all dictionary from your multiple
    resolutions.
    # For example:
    # results_all = {
    #     "results_Nx1": { "entryWallBottom": np.array([...]),
    "entryWallTop": np.array([...]), ... },
    #     "results_Nx2": { "entryWallBottom": np.array([...]),
    "entryWallTop": np.array([...]), ... },
    #     "results_Nx4": { ... },
    #     "results_Nx8": { ... },
    # }

    # Specify the prefix you want to plot.
    selected_prefix = "entryWallBottom"

    # Plot x vs pressure for the given prefix
    plot_coord_vs_pressure_across_res(results_all, selected_prefix,
    coord='x')

    selected_prefix = "entryWallTop"
    plot_coord_vs_pressure_across_res(results_all, selected_prefix,
    coord='x')

    selected_prefix = "stepFaceHoriz"
    plot_coord_vs_pressure_across_res(results_all, selected_prefix,
    coord='x')

    selected_prefix = "stepFaceVert"
    plot_coord_vs_pressure_across_res(results_all, selected_prefix,
    coord='y')

    # To plot y vs pressure instead, call:
    # plot_coord_vs_pressure_across_res(results_all, selected_prefix,
    coord='y')

```

Parallel Performance Plot

```
import numpy as np
import matplotlib.pyplot as plt

processors = np.array([1, 2, 4, 8, 16, 32, 56]) # P
actual_speedup = np.array([1.0, 2.150538, 4.223963, 8.039254, 13.36682,
21.37129, 26.80988])

x = np.linspace(1, 60, 200)
ideal_speedup = x          # Ideal = y = P
speedup_80 = 0.8 * x      # 80% efficient
speedup_60 = 0.6 * x      # 60% efficient

plt.figure(figsize=(8, 6))

# Plot actual
plt.plot(processors, actual_speedup, 'rs-', label='actual (1 M cells)')
# Plot the theoretical lines
plt.plot(x, ideal_speedup, 'k-', label='ideal')
plt.plot(x, speedup_80, 'k--', label='80% efficient')
plt.plot(x, speedup_60, 'k-.', label='60% efficient')

plt.xlim(0, 60)
plt.ylim(0, 70)
plt.xlabel('# of processors, P')
plt.ylabel('Speed-up,  $T(1)/T(P)$ ')
plt.title('Speed-up vs. Number of Processors')
plt.grid(True)
plt.legend(loc='upper left')
plt.show()
```